Diagrama de activitate si diagrama use-case - Comparatie

Diferența principală constă în scopul lor: **diagrama use-case** arată ce face sistemul din perspectiva utilizatorului, iar **diagrama de activitate** arată cum funcționează un proces pas cu pas. Ambele sunt diagrame comportamentale în UML (Unified Modeling Language), dar sunt folosite în etape diferite ale proiectării software.

Când folosești **diagrama Use-Case**

Această diagramă este ideală pentru a comunica cu clienții sau cu persoanele non-tehnice.

* Definirea limitelor sistemului: Arată clar ce face aplicația și ce rămâne în afara ei.

* Identificarea utilizatorilor: Stabilește rolurile (actorii) care interacționează cu sistemul, cum ar fi un client, un administrator sau un sistem extern de plăți.

* Exemplu concret: Într-un magazin online, diagrama arată că Clientul poate să Caute produse, să Adauge în coș și să Plătească comanda, fără a explica modul în care se face plata.

Când folosești **diagrama de Activitate**

Această diagramă seamănă foarte mult cu o organigramă (flowchart) și este utilă pentru dezvoltatori și analiști de business.

* Modelarea logicii complexe: Arată exact ce se întâmplă când o condiție este adevărată sau falsă.

* Procese paralele: Poate ilustra acțiuni care se întâmplă simultan, cum ar fi trimiterea unui e-mail de confirmare în același timp cu actualizarea stocului.

* Exemplu concret: Detaliază cazul de utilizare Plătește comanda. Arată introducerea datelor cardului, verificarea fondurilor, decizia dacă plata a reușit sau a eșuat și afișarea mesajului corespunzător.

Cum lucrează împreună

De cele mai multe ori, cele două diagrame se completează reciproc. Mai întâi creezi o diagramă use-case pentru a mapa toate funcționalitățile aplicației. Apoi, pentru fiecare elipsă (caz de utilizare) mai complexă, desenezi câte o diagramă de activitate pentru a explica algoritmul din spate

Tabel comparativ

Criteriu	**Diagrama Use-Case (Cazuri de utilizare)**	**Activity Diagram** (Diagrama de Activitate)
Întrebarea la care raspunde	**Ce** funcționalități oferă sistemul si **cine** le foloseste?	**Cum** se desfășoară un flux de lucru sau procesul în detaliu?
Nivelul de detaliu	Ridicat - Privire de ansamblu asupra cerintelor	Scazut - Detalii interne, pasii exacti,ramificatii, bucle ,logica
Perspectiva	Externă (ce vede utilizatorul final).	Internă (fluxul logic și operațional).
Ordinea în timp	Nu arată o ordine cronologică a acțiunilor.	Arată secvența clară a pasilor și execuția în paralel.
Elemente principale	Actori (omuleti), cazuri de utilizare (elipse), relații.	Acțiuni, decizii (romburi), sageti de flux, noduri de start/final.
Cand se foloseste	La începutul proiectului, pentru a stabili cerințele cu clientul.	In etapa de analiza si design, pentru a detalia logica unui caz de utilizare

Diagrama de activitate si diagrama logica – Comparatie

Diagrama de activitate ilustrează fluxul dinamic al proceselor și acțiunilor dintr-un sistem, în timp ce **diagrama logică** reprezintă pașii decizionali și structura unui algoritm.

Ambele sunt instrumente vizuale esențiale în dezvoltarea software și analiza de business, dar au scopuri și niveluri de detaliu diferite.

Când să le folosești

- **Alege diagrama de activitate** atunci când vrei să arăți cum colaborează diferite departamente sau module software. Este ideală pentru procese complexe cu acțiuni care se desfășoară în același timp.
- **Alege diagrama logică** atunci când proiectezi logica internă a unei funcții sau a unui script. Este foarte utilă pentru a explica rapid reguli simple de tipul „dacă-atunci” (*if-else*) și bucle repetitive.

Tabel comparativ

Criteriu	**Diagrama de activitate (UML)**	**Diagrama logică (Flowchart)**
Scop principal	Modelează procese de business, fluxuri de lucru și comportamentul sistemului.	Reprezintă logica unui algoritm, a unui program sau a unei proceduri simple.
Nivel de abstractizare	Ridicat sau mediu, orientat spre procese și interacțiuni între componente.	Scăzut și detaliat, orientat spre instrucțiuni pas cu pas.
Procesare paralelă	Suportă nativ acțiuni simultane prin bare de sincronizare (*fork/join*).	Nu suportă nativ paralelismul, urmând o execuție strict secvențială.
Responsabilități	Poate împărți acțiunile pe roluri sau departamente folosind benzi (*swimlanes*).	Nu indică cine execută acțiunea, ci doar ce se întâmplă în algoritm.